

AI-Driven LLVM IR Mutation and Differential Testing for Compiler Bug Discovery

Sumadhva Krishna H M, Sumedh Udapa U, Suryanarayana M *Department of Computer Science and Engineering
RV College of Engineering*

Bengaluru, India

sumadhvak.cs23@rvce.edu.in, sumedhu.cs23@rvce.edu.in, suryanarayanm.cs23@rvce.edu.in

Abstract—This study investigates whether large language models can generate useful LLVM IR mutants for differential compiler testing. We present a compiler-testing pipeline that mutates LLVM IR seeds, filters candidates through structural verification, and compares validated programs across optimization levels. The implementation combines a FastAPI backend, a React dashboard, and Dockerized LLVM and Ollama services. Seeds are uploaded as .ll files, mutated through three strategies (LLM-guided, grammar-based, and random), validated with `llvm-as` and `opt -passes=verify -disable-output`, and then compared under `-O0` and `-O2` to classify compiler behavior. The system records generation metadata, validity outcomes, invalid-mutant taxonomies, study runs, and differential results in JSON and CSV logs so that validity rate, mismatch rate, duplicate rate, and failure categories can be reported consistently.

Index Terms—LLVM IR, compiler fuzzing, differential testing, mutation testing, Ollama, FastAPI, Docker, validation, manifest tracking

I. INTRODUCTION

LLVM IR fuzzing is a practical way to stress compiler front ends, optimizers, and code generators with syntactically valid but semantically unusual programs. This study examines a reproducible pipeline for seed ingestion, mutant generation, validity filtering, and differential execution. The central question is whether LLM-guided mutation can complement grammar-driven and random baselines by producing valid, non-trivial IR that exposes compiler behavior under optimization.

The motivation for this work is twofold. First, compiler fuzzers often generate large numbers of candidates, but many are either invalid or too close to the original seed to be useful. Second, LLMs can propose structurally novel program fragments, yet they are also prone to violating SSA, control-flow, and type constraints. A useful LLVM fuzzing framework therefore needs both generation diversity and rigorous filtering.

The remainder of this paper is organized as follows. Section II reviews related work and identifies the gap addressed by this study. Section III describes the methodology, including mutation, validation, and differential execution. Section IV presents the observed results and discussion. Section V discusses threats to validity, and Section VI concludes the paper.

II. RELATED WORK

A. Grammar-Guided LLVM Fuzzing

IRFuzzer [1] is one of the strongest LLVM backend fuzzers in recent literature. It relies on grammar- and structure-guided

mutation to preserve validity while exploring control flow, vector types, and function definitions. Its reported bug-finding results show that valid-IR-first generation is highly effective, but the approach does not study how LLM-generated mutations compare with grammar-guided edits in terms of validity, semantic interest, or differential-testing yield.

B. LLM-Based Compiler Testing

LegoFuzz [2] demonstrates that large language models can be used in compiler testing when generation is separated from validation. The work uses offline generation and online verification to reduce compute cost and has shown that LLM-generated programs can uncover optimization bugs. However, it focuses more on source-level or high-level IR-like programs than on fine-grained LLVM IR instruction mutation, leaving room for LLVM-specific mutation studies.

C. LLMs and Compiler Understanding

The survey by [3] shows that LLMs are increasingly used across compiler tasks, including IR understanding, optimization suggestions, and test generation. At the same time, it emphasizes that LLMs still struggle with structural correctness such as SSA, dominance, and control-flow invariants. This aligns closely with the need for verifier-based filtering in LLVM IR mutation workflows.

The study by [4] further reinforces this point by showing that even strong models can miss control-flow edges or generate invalid CFGs when asked to interpret LLVM IR. The work motivates symbolic validation alongside LLM output, but it does not evaluate a mutation pipeline that compares LLM-generated IR directly against grammar-based mutants in a differential-testing setup.

D. LLM-Assisted IR Transformation

Intent-driven IR optimization with LLMs [5] shows that LLMs can be used to rewrite IR passes and generate optimization-intent-driven IR variants. The work also demonstrates that LLMs can synthesize fuzzing harnesses for IR-level differential testing. Its setting, however, assumes valid IR transformation contexts rather than unconstrained mutation, so it does not address validity breakdowns in raw LLM-generated LLVM IR.

E. Hybrid and Feedback-Guided Mutation

Hybrid fuzzing with LLM-guided input mutation and semantic feedback [6] shows that LLMs can be steered by analysis feedback to improve coverage and bug-finding. The paper is valuable as a methodology reference, but it targets program inputs rather than LLVM IR instruction mutation. The same is true for the broader mutation-testing study in [7], which finds that LLM-generated mutants can be behaviorally strong but evaluates source-level mutation rather than LLVM IR.

F. Empirical LLVM IR Studies

The empirical LLVM IR study by [8] is especially relevant because it evaluates LLMs on IR interpretation, optimization, and translation using a verification pipeline. Its results show that LLMs can produce non-trivial IR transformations, but many variants are still invalid or low-value. That provides a strong empirical basis for the present study, which focuses more narrowly on mutation quality and differential-testing behavior.

G. Research Gap

Taken together, the literature shows three gaps that motivate this study. First, most prior work is either grammar-based or LLM-based, but not a direct LLVM-IR-level comparison of both mutation styles under the same validation and differential-testing pipeline. Second, the literature rarely categorizes invalid LLVM IR mutants by concrete verifier failure types such as SSA, PHI dominance, type errors, or CFG violations. Third, most existing work does not combine mutation, validation, manifest tracking, and differential testing into one reproducible study framework.

III. METHODOLOGY

A. Pipeline Overview

Our implementation centers on a FastAPI backend that exposes routes for seeds, mutants, differential execution, and analysis. The pipeline is organized into three stages. Seeds are ingested as LLVM IR files, mutants are generated using one of three strategies, and the resulting programs are filtered through LLVM-based verification before differential testing is performed on the valid subset.

The code is organized into route, service, model, and utility layers to keep the mutation workflow modular. Raw generation metadata is written to `raw_mutants.json`, validation outcomes are recorded in `validity_logs.json`, and the analysis layer aggregates these logs into a manifest for downstream comparison. This separation allows the study to preserve traceability without introducing a heavy database dependency.

B. Seed Corpus

Seeds are stored as LLVM IR `.ll` files and can be listed through `GET /api/v1/seeds`. The corpus serves as a fixed baseline so that all mutation strategies are evaluated

on the same input families. In the current project, the seed corpus is treated as an experimental control rather than as a dynamic dataset, which makes comparisons across mutators more interpretable.

C. Mutation Strategies

The repository implements three mutation modes through `POST /api/v1/mutants/generate`. The LLM-guided strategy uses Ollama to produce a complete mutated LLVM IR module from a prompt that asks for exactly one change. The grammar-based strategy applies deterministic rule-based transformations designed to preserve syntactic validity. The random strategy provides a stochastic baseline for comparison and ablation-style analysis.

The LLM mutator uses prompt constraints that emphasize SSA correctness, complete module output, and valid LLVM syntax. It also supports refinement loops when generation errors need to be corrected with error feedback. This design reflects the central tension in LLM-based compiler testing: the model must be creative enough to change the program, but constrained enough to preserve LLVM invariants.

D. Deduplication and Logging

Before a mutant is written to disk, its normalized IR hash is checked against previously seen hashes so that duplicate candidates can be skipped early. Every generated mutant is still recorded in `raw_mutants.json` with metadata such as seed name, mutator type, generation strategy, file path, and content hash. This preserves the generation history even when some candidates are filtered out later.

E. Structural Validation

The validation stage is exposed through `POST /api/v1/mutants/validate`. Each mutant is checked in an isolated worker process to keep one broken file from poisoning the whole batch. The validator runs `llvm-as` first and then `opt -passes=verify -disable-output` on the compiled bitcode. Invalid mutants are moved into `invalid_mutants/`, valid mutants are moved into `valid_mutants/`, and the per-mutant result is appended to `validity_logs.json`.

The validation log captures whether the mutant was valid, the verifier output, the error type, whether the file was a duplicate, whether the output was semantically trivial, and whether a timeout occurred. The error taxonomy is intentionally compact so that the final analysis can summarize failures without manual relabeling.

F. Differential Execution

The differential tester is exposed through `POST /api/v1/differential/run` and `GET /api/v1/differential/results`. It works on the set of valid mutants and compares compiler behavior at two optimization settings, defaulting to `-O0` and `-O2`. The request can also target a subset of mutant IDs and can cap the number of mutants tested in one run.

The implementation follows a simple but practical execution policy. If a mutant contains `@main`, it is compiled and executed directly. If `@main` is absent, the service attempts to discover a callable entry function and builds a small harness around it. Differential outcomes are written to `results.csv` with fields for the mutant ID, optimization pair, mismatch type, execution mode, failure stage, harness entry, runtimes, and run ID.

IV. RESULTS AND DISCUSSION

The analysis endpoints convert the study logs into publishable summaries. The main route group is `GET /api/v1/analysis/invalid-taxonomy`, `POST /api/v1/analysis/run-study`, `GET /api/v1/analysis/seed-sensitivity`, `GET /api/v1/analysis/study-history`, and `GET /api/v1/analysis/manifest`. These endpoints support the comparison tables and figures needed for the final paper.

Invalid mutants are categorized from verifier output into labels such as `broken_ssa`, `invalid_phi_dominance`, `type_error`, `syntax_parse`, `cfg_error`, and `other_verifier_error`. This taxonomy captures the dominant LLVM failure modes observed in the pipeline and provides a concise way to summarize where LLM-generated mutants break down.

The controlled-study endpoint executes an experiment matrix over seed names, count per seed, mutator types, and optimization pairs. For each configuration it generates mutants, validates them, and optionally runs differential testing on the valid subset. This design supports direct comparison of mutation strategies under identical execution conditions and yields aggregate metrics such as validity rate and mismatch rate over valid mutants.

The manifest layer combines generation and validation logs into a single structured snapshot. Each entry stores the mutant ID, seed name, mutator type, mutation strategy, validity, error type, content hashes, path, duplicate status, and triviality flag. The summary section keeps counts by mutator type and error type. This structured snapshot is the primary artifact used for the comparison tables and consistency checks in the final paper.

The project documentation indicates that Phase 3 validation produced 111 real mutants in a snapshot and that unit tests passed at 100 percent. At the study level, the documented patterns are consistent with prior literature: grammar-based mutation tends to preserve validity better, while LLM-based mutation is more likely to introduce novel but structurally fragile IR. The most interesting cases are valid mutants that are not trivially equivalent, because they combine correctness with behavioral variation.

A. Empirical Evaluation

The reported metrics are validity rate, mismatch rate, duplicate rate, triviality rate, and failure counts by category. These metrics are appropriate because they separate generation quality from execution behavior. The evaluation therefore

measures not just whether a candidate passes verification, but also whether it survives differential testing and whether it produces meaningful variation.

V. THREATS TO VALIDITY

The current prototype still has clear limitations. It relies on a local or containerized Ollama service, so model availability and response quality affect generation quality. It uses flat-file logs rather than a database, which is sufficient for the prototype but may become harder to manage as the corpus grows. It also treats semantic-triviality detection as a secondary filter rather than a complete semantic equivalence checker.

A second threat is corpus size. The current study is intentionally narrow, which makes it suitable as a controlled prototype but limits statistical generalization. Broader seed corpora and longer experimental runs would improve confidence in the observed trends. A third threat is that compiler behavior is inherently stochastic under optimization and runtime conditions, so some mismatch categories may shift as the environment changes.

Future work should focus on stronger semantic novelty checks, richer mismatch classification, larger seed corpora, broader optimization pairs, and more automated reporting. A larger study could also compare additional LLMs or integrate coverage-oriented metrics to quantify exploration quality rather than only syntactic validity.

VI. CONCLUSION

This study presents a complete LLVM IR fuzzing pipeline centered on seed management, LLM and baseline mutation, LLVM-based validity filtering, differential execution, and structured analysis. The manifest and study logs provide the evidence trail for reporting results, while the frontend and API layers make the system practical to operate during experiments. The paper now reflects the actual compiler-testing study rather than an unrelated domain example or implementation sketch.

REFERENCES

- [1] J. Rong, et al., “IRFuzzer: Specialized Fuzzing for LLVM Backend Code Generation,” arXiv preprint arXiv:2402.05256, 2024.
- [2] “LegoFuzz: Interleaving Large Language Models for Compiler Testing,” arXiv preprint arXiv:2508.18955, 2025.
- [3] “A Survey on the Synergy of LLMs and Compilers,” arXiv preprint arXiv:2601.02045, 2026.
- [4] “Can Large Language Models Understand Intermediate Representations?” arXiv preprint arXiv:2502.06854, 2025.
- [5] “Intent-Driven IR Optimization with Large Language Models,” arXiv preprint arXiv:2602.18511, 2026.
- [6] “Hybrid Fuzzing with LLM-Guided Input Mutation and Semantic Feedback,” arXiv preprint arXiv:2511.03995, 2025.
- [7] “Large Language Models for Mutation Testing,” arXiv preprint arXiv:2406.09843, 2024.
- [8] “Evaluating the Ability of LLMs to Interpret, Optimize and Translate LLVM IR,” Kennesaw State University study, 2025.
- [9] LLVM Project, “LLVM Language Reference Manual,” <https://llvm.org/docs/LangRef.html>.
- [10] LLVM Project, “llvm-as – LLVM assembler,” <https://llvm.org/docs/CommandGuide/llvm-as.html>.
- [11] LLVM Project, “opt – LLVM optimizer,” <https://llvm.org/docs/CommandGuide/opt.html>.
- [12] FastAPI, “FastAPI Documentation,” <https://fastapi.tiangolo.com/>.

- [13] Ollama, “API Documentation,” <https://github.com/ollama/ollama/blob/main/docs/api.md>.
- [14] Docker, “Docker Compose Documentation,” <https://docs.docker.com/compose/>.